Notas de Aula
Prof. Adriano Milanez

Programação Web I
Responsive Web Development
Front-End Design Engineering
Programação Web Responsiva

[aula 17]
[Atualizado em: 14/08/2025]

[VITE – Componentização]

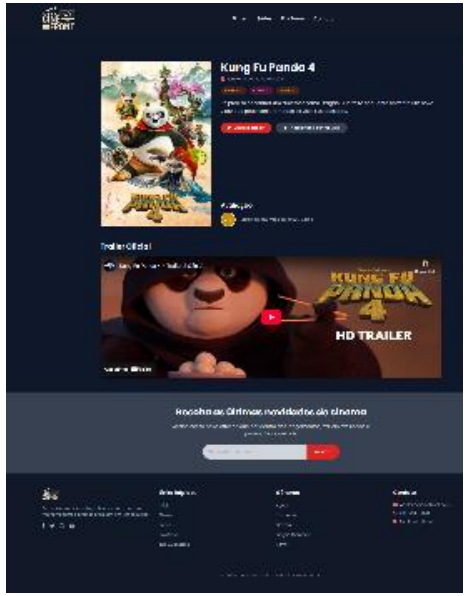
1

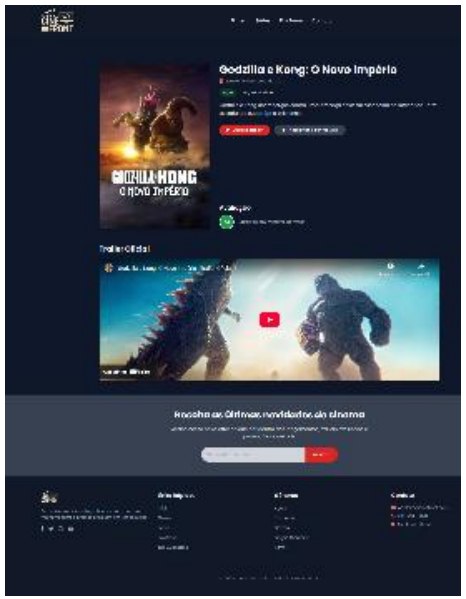
VITE – COMPONENTIZAÇÃO

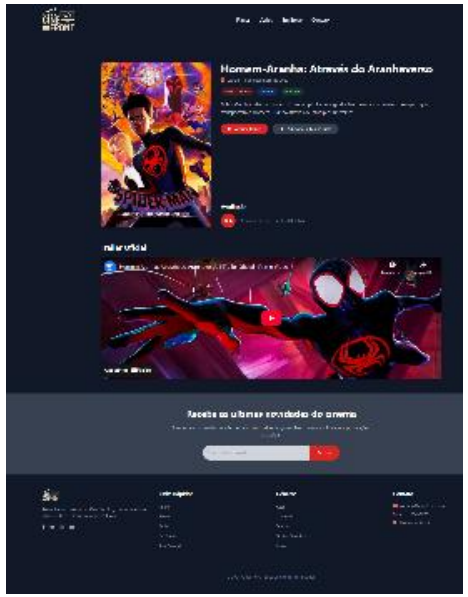
COMPONENTIZAÇÃO

- **O que é Componentização?**

  - Se precisássemos modificar algo no Cabeçalho ou no Rodapé das páginas ao lado, teríamos de acessar o código-fonte de cada uma das páginas e fazer as modificações necessárias.
  - Neste exemplo, temos apenas QUATRO páginas e se tivéssemos inúmeras páginas? Quantas vezes teríamos de modificar? Quanto tempo levaríamos para modificar?
  - E se fizéssemos essa modificação apenas UMA ÚNICA vez? Seria mais fácil?
  - Por isso vamos dividir nosso código em pequenas partes chamadas de **COMPONENTES**!







COMPONENTIZAÇÃO

- **O que é Componentização?**

  - Componentização é a prática de dividir uma interface em partes menores e reutilizáveis chamadas componentes.

  - Cada componente representa uma "peça" da interface, como um botão, cabeçalho, card de produto, menu, etc.

  - Em vez de escrever uma única página com todo o HTML e lógica misturados, usamos componentes para separar visualmente e funcionalmente cada parte da aplicação. Isso torna o código mais limpo, reutilizável e fácil de manter.

COMPONENTIZAÇÃO

- **Por que usar componentes?**
  - **Reutilização:** você pode usar o mesmo botão ou card em várias páginas sem reescrever o código.
  - **Organização:** o projeto fica dividido em arquivos pequenos e fáceis de entender.
  - **Clareza:** cada componente tem uma única responsabilidade (princípio SRP - Single Responsibility Principle).
  - **Escalabilidade:** conforme o aplicação cresce, é mais fácil manter e modificar.

COMPONENTIZAÇÃO

- ✓ **Diferença entre Componentes x Páginas?**

  - ✓ **COMPONENTES**
    - ✓ **Um componente** é uma **função isolada** que representa uma parte da interface (UI).
    - ✓ São pedaços isolados e reutilizáveis.
    - ✓ Função pequena e reutilizável
      - ✓ <Botao />, <Header />, <Card />
  - ✓ **PÁGINAS**
    - ✓ **Uma página** é um **agrupamento de componentes**, estruturada como uma tela completa e geralmente ligada a uma **rota** da aplicação.
    - ✓ São montadas **usando vários componentes** e representam **uma rota completa** da aplicação.
    - ✓ Função maior que organiza vários componentes para formar uma tela
      - ✓ <Home />, <Contato />, <Produtos />

COMPONENTIZAÇÃO

- ✓ **Diferença entre Componentes x Páginas?**

Característica	Componente	Página
Função	Parte da interface	Tela completa (rota)
Reutilização	Alta (em várias páginas)	Geralmente única
Local no projeto	src/components/	src/pages/
Exemplo	Botao, Header, Card	Home, Contato, Produtos
Uso em rotas	Não diretamente	Sim, via <Route path="/..." element={<Pagina />} />

COMPONENTIZAÇÃO

- **Componentização com TypeScript**

  - O TypeScript adiciona um nível adicional de segurança e clareza ao desenvolvimento React, fornecendo tipagem estática. Aqui estão algumas práticas recomendadas ao combinar componentização com TypeScript:

  - **Tipagem Forte:** Aproveite ao máximo a tipagem estática do TypeScript para garantir que os Componentização sejam usados corretamente e para evitar erros durante a compilação.

```
interface Props {
  nome: string;
  idade: number;
}

const MeuComponente: React.FC<Props> = ({ nome, idade }) => {
  // ...
};
```

COMPONENTIZAÇÃO

- ✓ **Componentização com TypeScript**

  - ✓ **Propriedades Opcionais e Padrão:** Use propriedades opcionais ou valores padrão para tornar os Componentização mais flexíveis e fáceis de usar.

```
interface BotaoProps {
  cor?: string;
  tamanho?: 'pequeno' | 'médio' | 'grande';
}

const Botao: React.FC<BotaoProps> = ({ cor = 'azul', tamanho = 'médio' }) => {
  // ...
};
```

COMPONENTIZAÇÃO

- ✓ **Componentização com TypeScript**

  - ✓ **Hooks e Lógica Encapsulada:** Separe a lógica do componente usando hooks e funções auxiliares, tornando cada componente mais focado e fácil de entender.

```
const MeuComponente: React.FC<Props> = ({ nome, idade }) => {
  const saudacao = `Olá, ${nome}!`;

  useEffect(() => {
    console.log(`Idade: ${idade}`);
  }, [idade]);

  return <div>{saudacao}</div>;
};
```

COMPONENTIZAÇÃO

- ✓ **Componentização com TypeScript**

  - ✓ **Composição Inteligente:** Use a composição de Componentização para criar interfaces complexas, combinando vários Componentização menores.

```
const PaginaPerfil: React.FC<Props> = ({ nome, idade }) => {
  return (
    <div>
      <Cabecalho nome={nome} />
      <MeuComponente nome={nome} idade={idade} />
      <Rodape />
    </div>
  );
};
```

COMPONENTIZAÇÃO

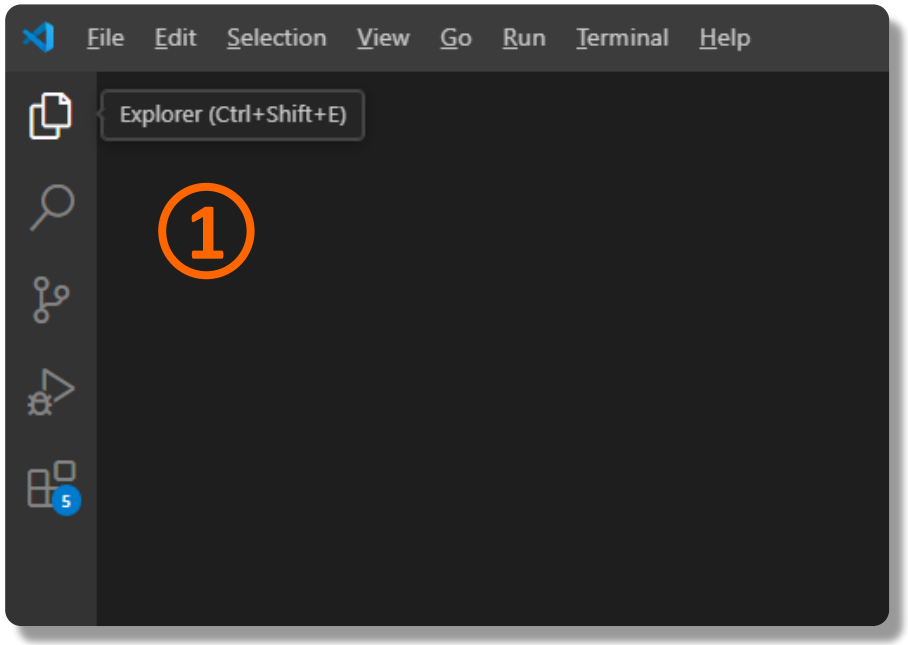
- **Componentização com TypeScript**

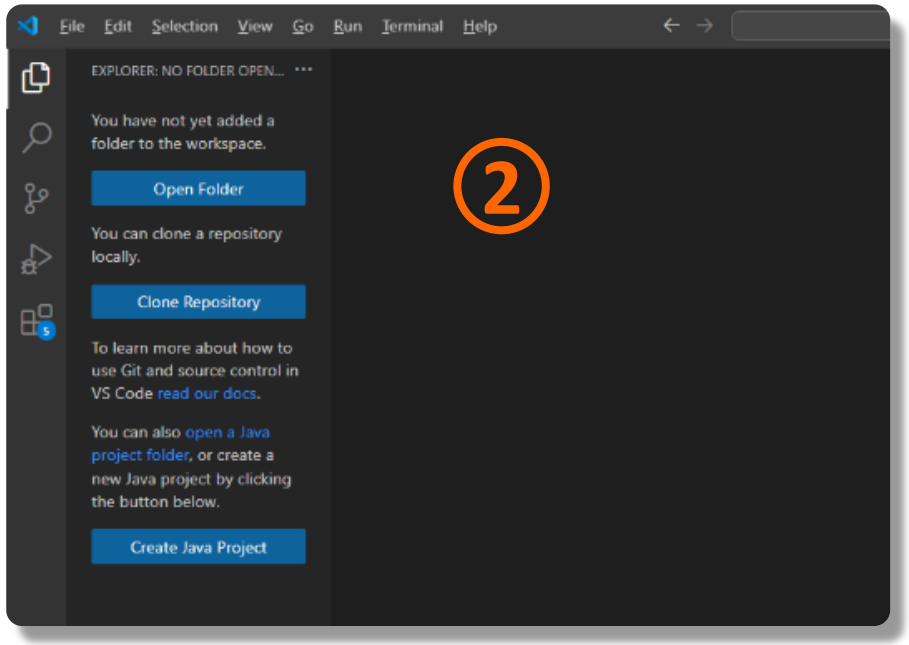
  - Ao adotar a componentização em React com TypeScript, os desenvolvedores podem criar aplicações mais flexíveis, escaláveis e fáceis de manter. A capacidade de criar Componentização reutilizáveis, encapsular lógica de maneira eficiente e aproveitar a tipagem estática do TypeScript contribui para um desenvolvimento mais robusto e eficiente. Ao seguir as práticas recomendadas, os projetos podem alcançar um equilíbrio ideal entre modularidade, clareza e desempenho.

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.
    1. Selecionamos o ícone ‘Explorer’
    2. Na sequência, clique em ‘Open folder’



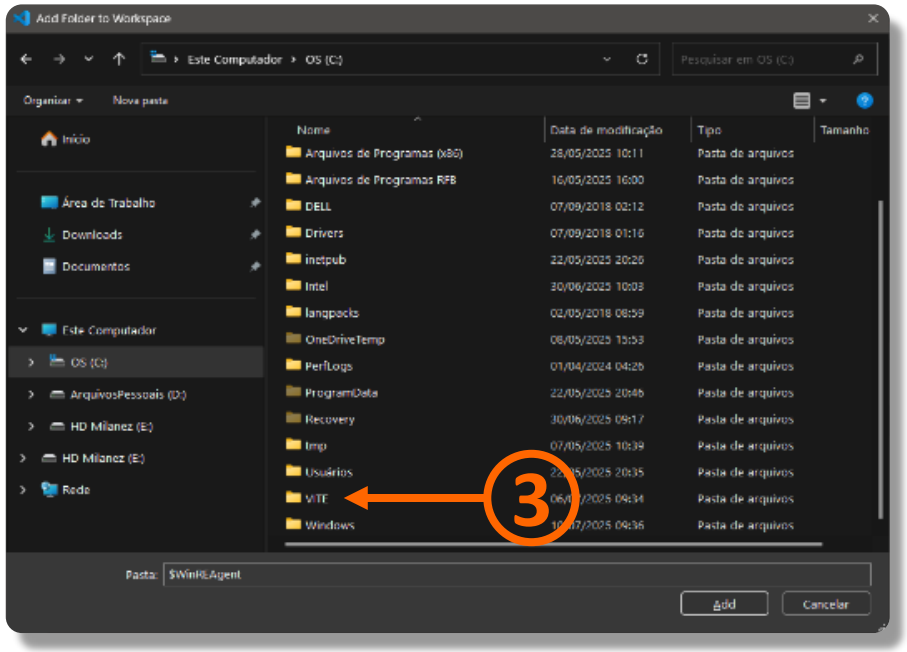


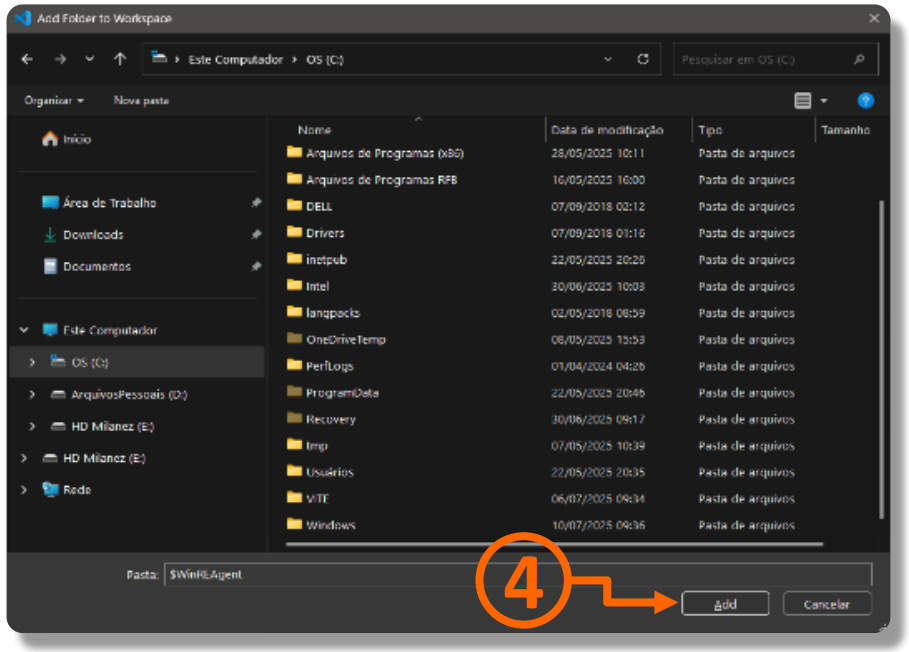
COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    3. Procure a pasta ‘VITE’, que criamos na aula anterior, selecione
    4. Na sequência clique em ‘Add’.



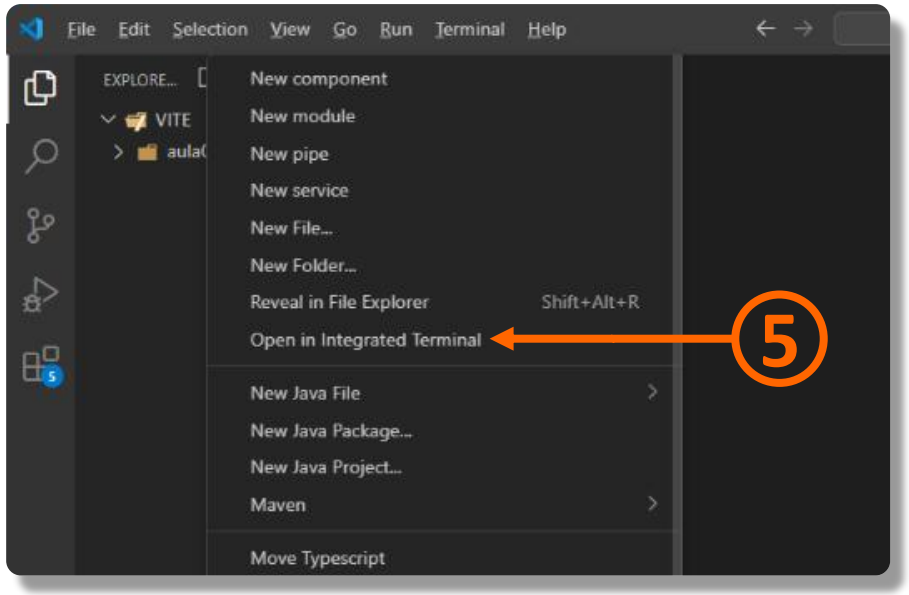


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    5. Em uma área qualquer sem conteúdo, clicamos com o botão direito do mouse e selecionamos ‘Open in Integrated Terminal’

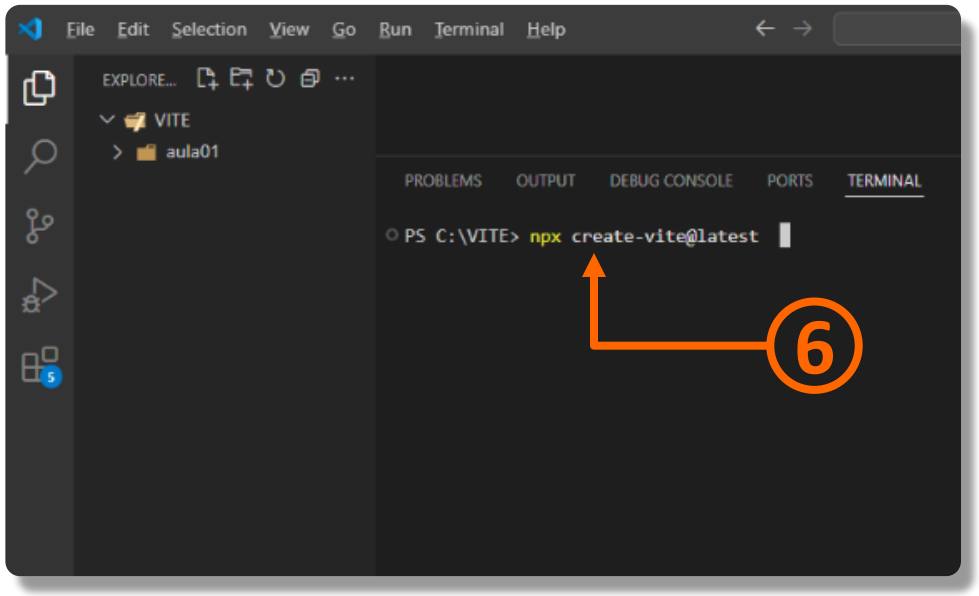


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    6. Na sequência digitamos a instrução ‘`npx create-vite@latest`’

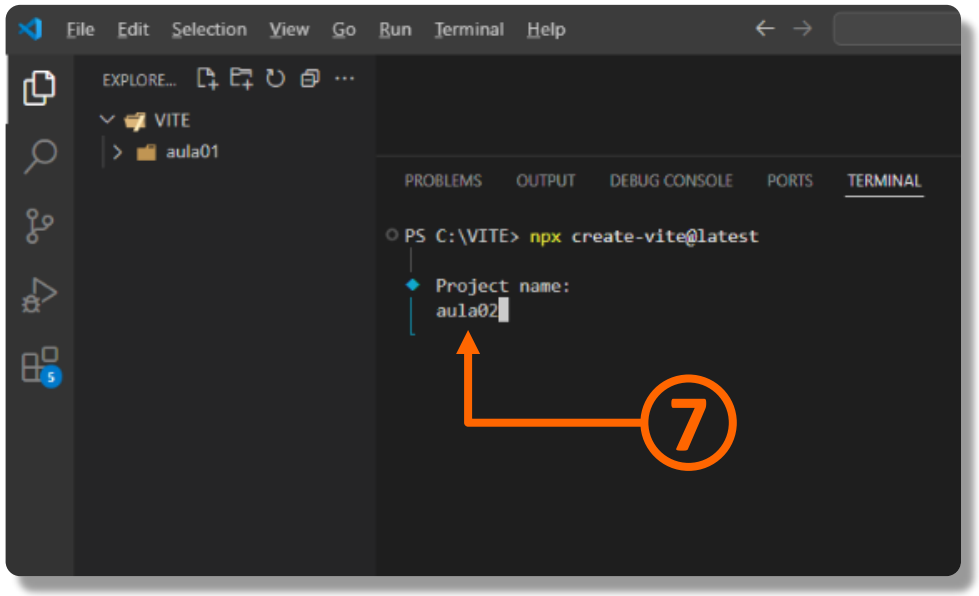


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    7. Na sequência damos o nome ao nosso projeto ‘`aula02`’, lembrando das regras de nomenclatura que devemos seguir.

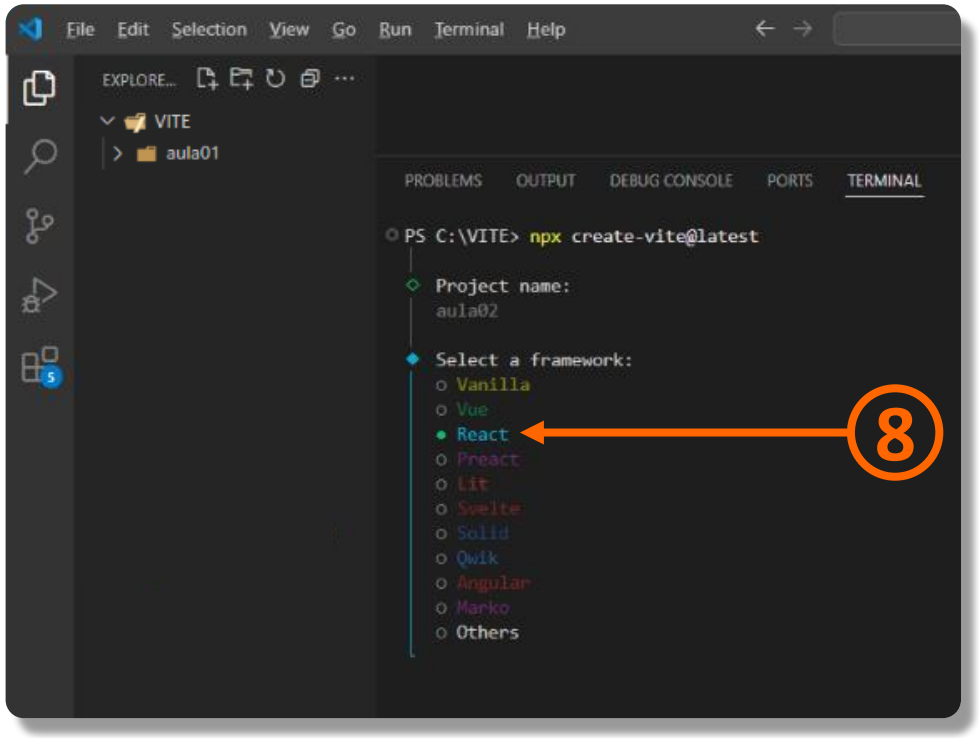


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    8. Selecionamos o *framework* que vamos trabalhar, no nosso caso, '`React`'.

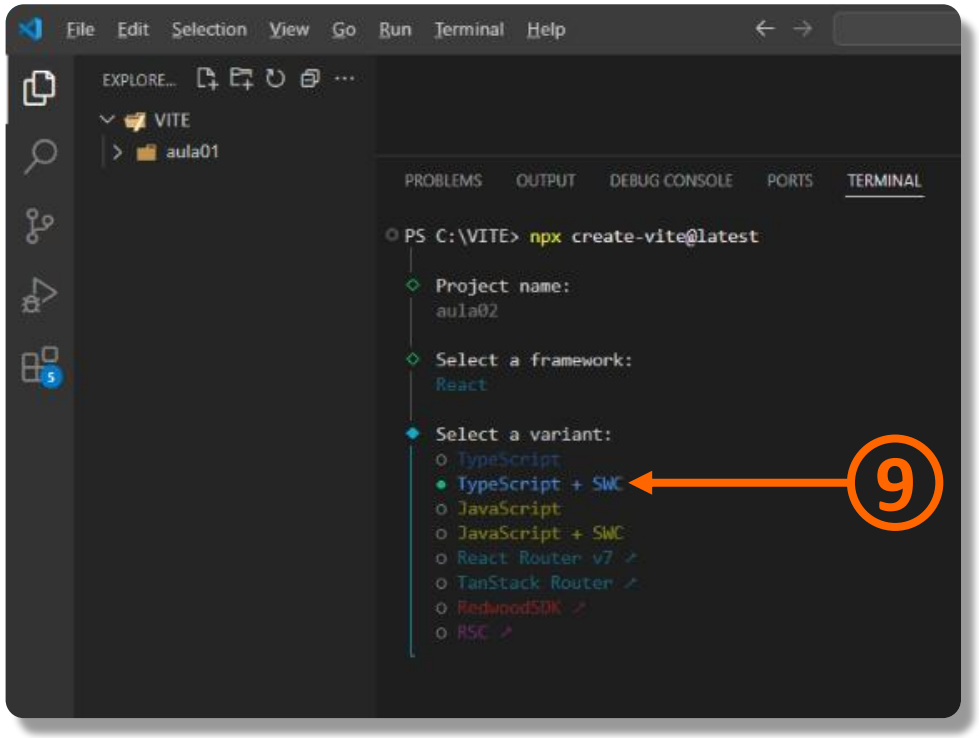


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    9. Selecionamos a *variant*, no nosso caso, '`TypeScript + SWC`'.

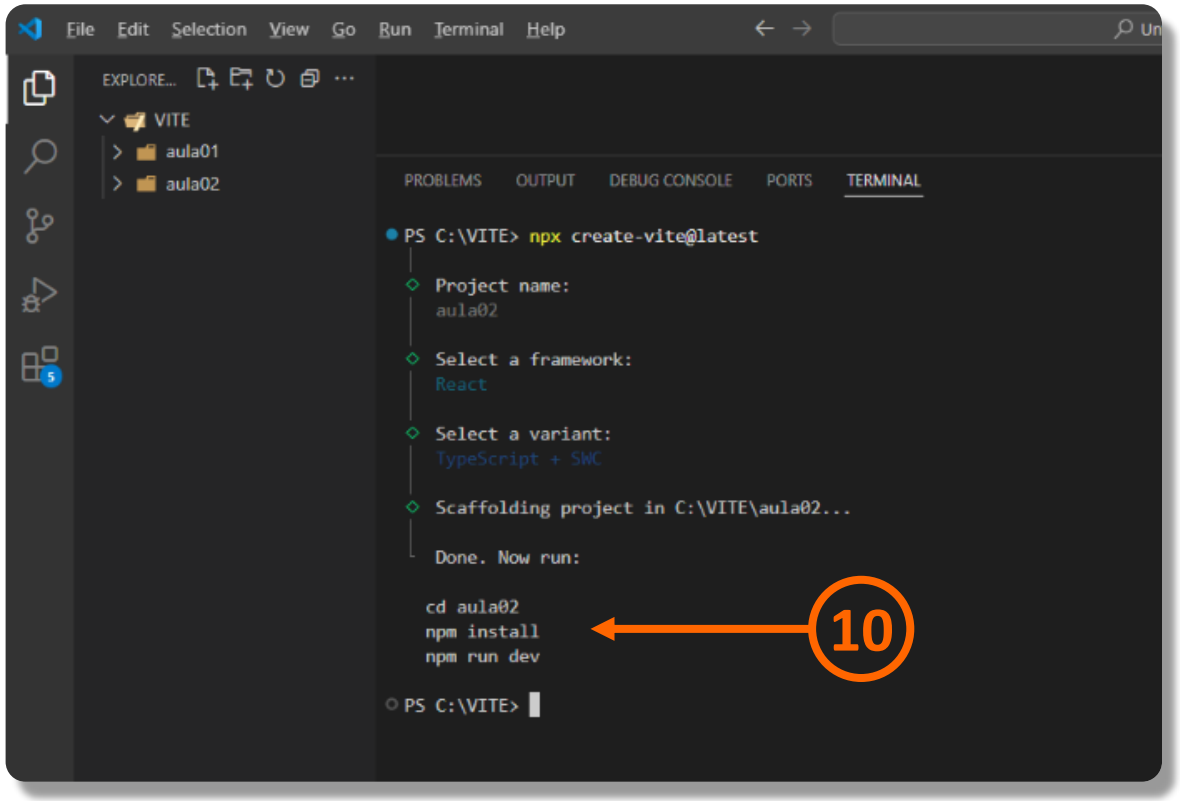


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    10. Após a criação do projeto teremos a seguinte resposta.

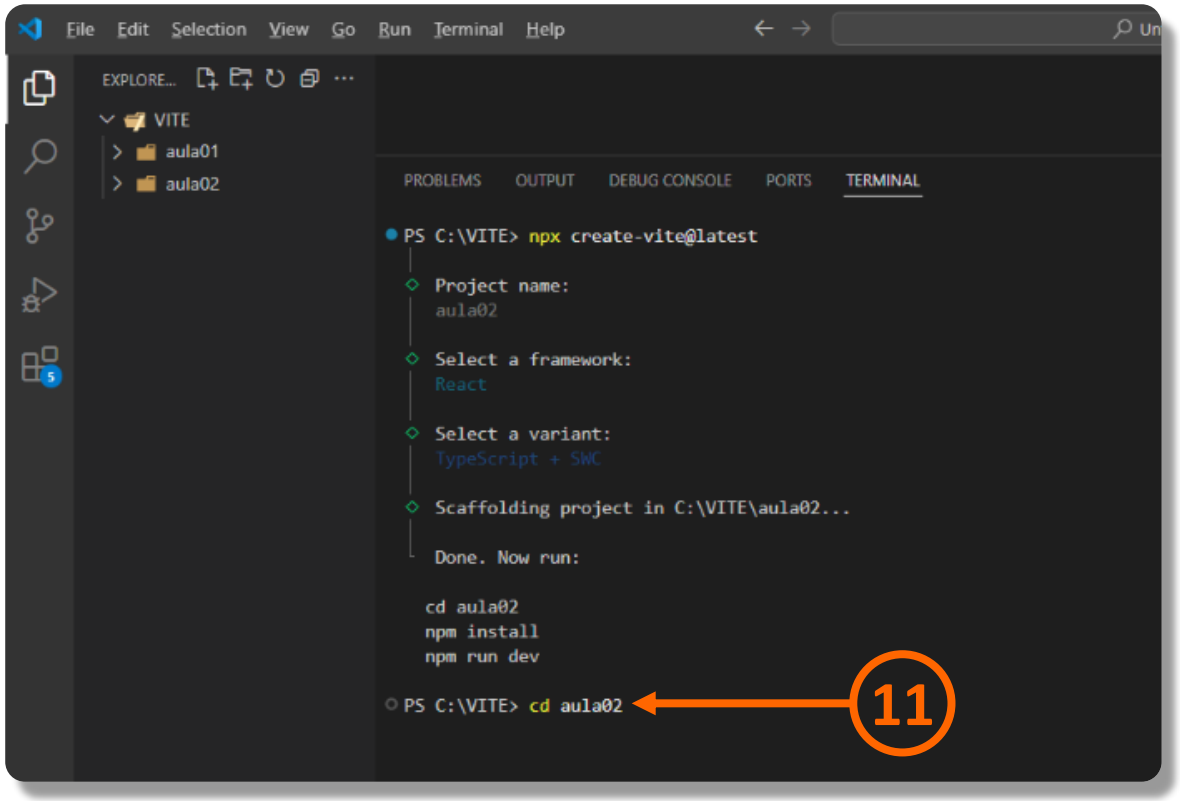


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    11. Como na imagem, vamos fazer o que se pede, no prompt digitamos '`cd aula02`'.

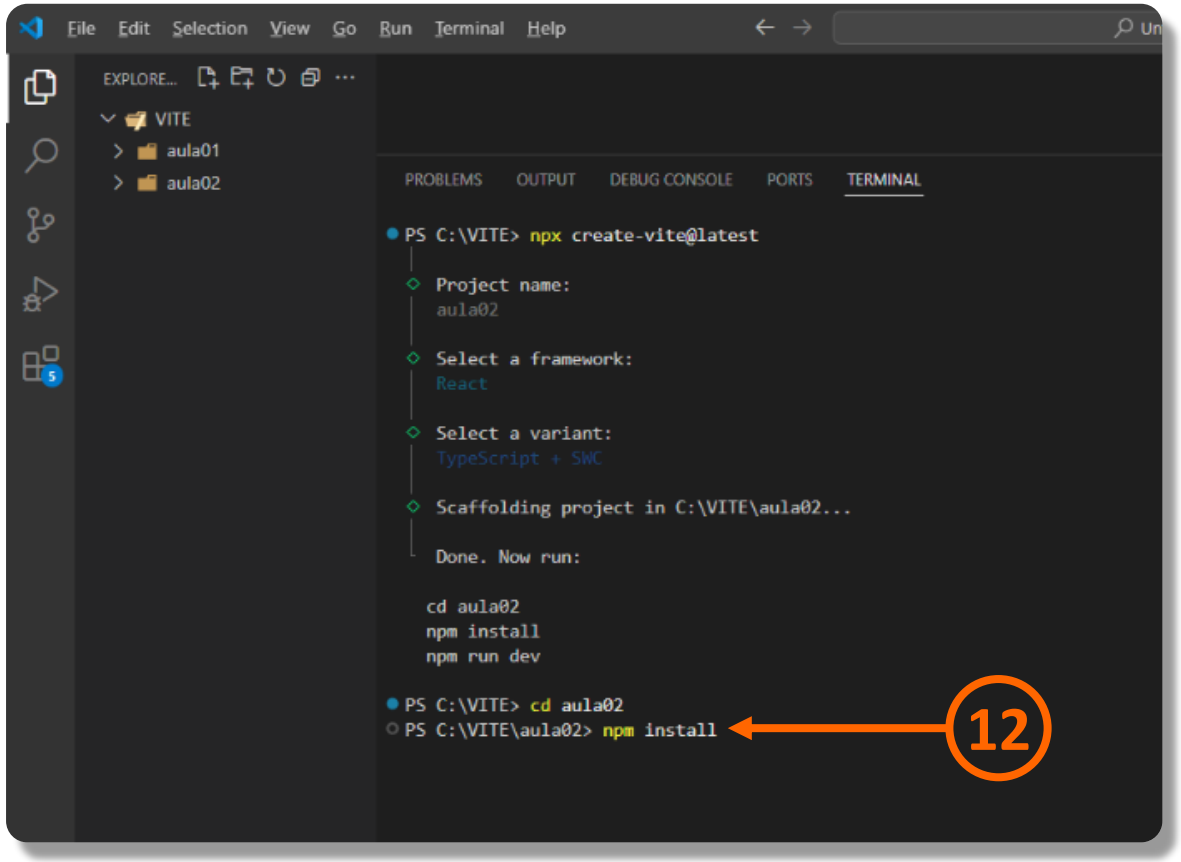


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    12. Como na imagem, vamos fazer o que se pede, no prompt digitamos ‘`npm install`’.

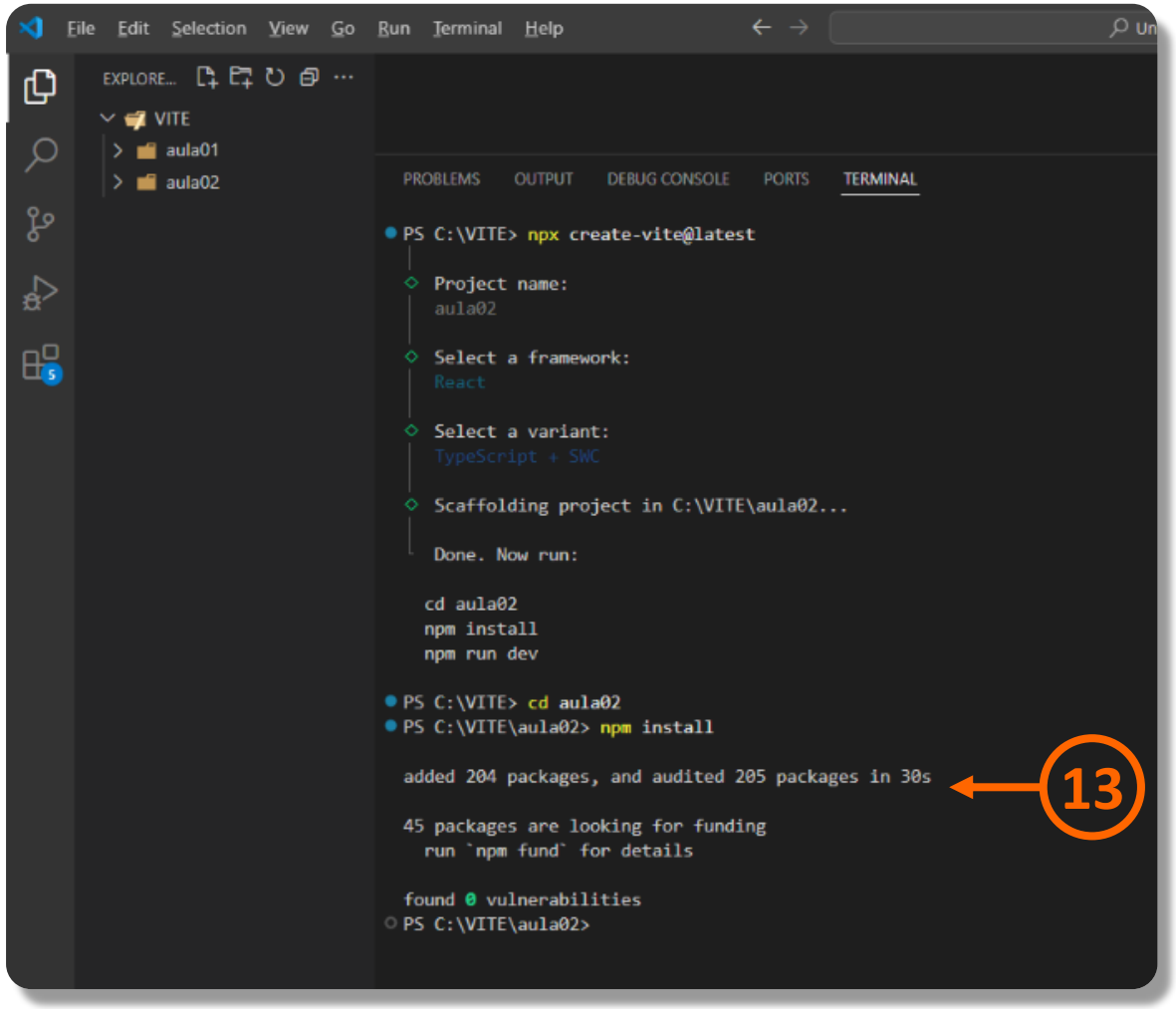


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    13. Como resultado temos a instalação dos pacotes necessários para que nossa aplicação ‘rode’.

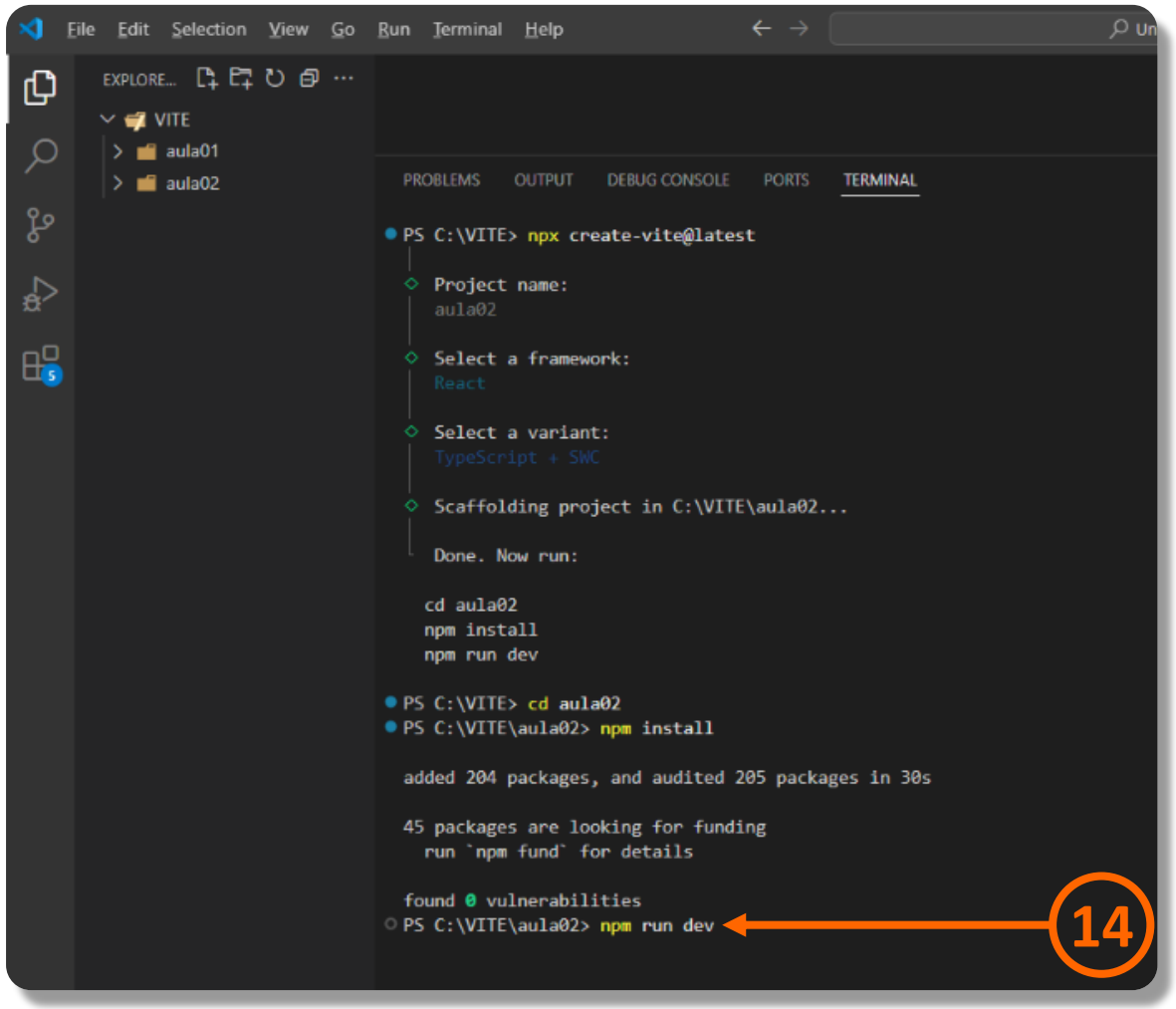


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    14. Vamos digitar no prompt ‘`npm run dev`’ para que consigamos ‘rodar’ nossa aplicação.

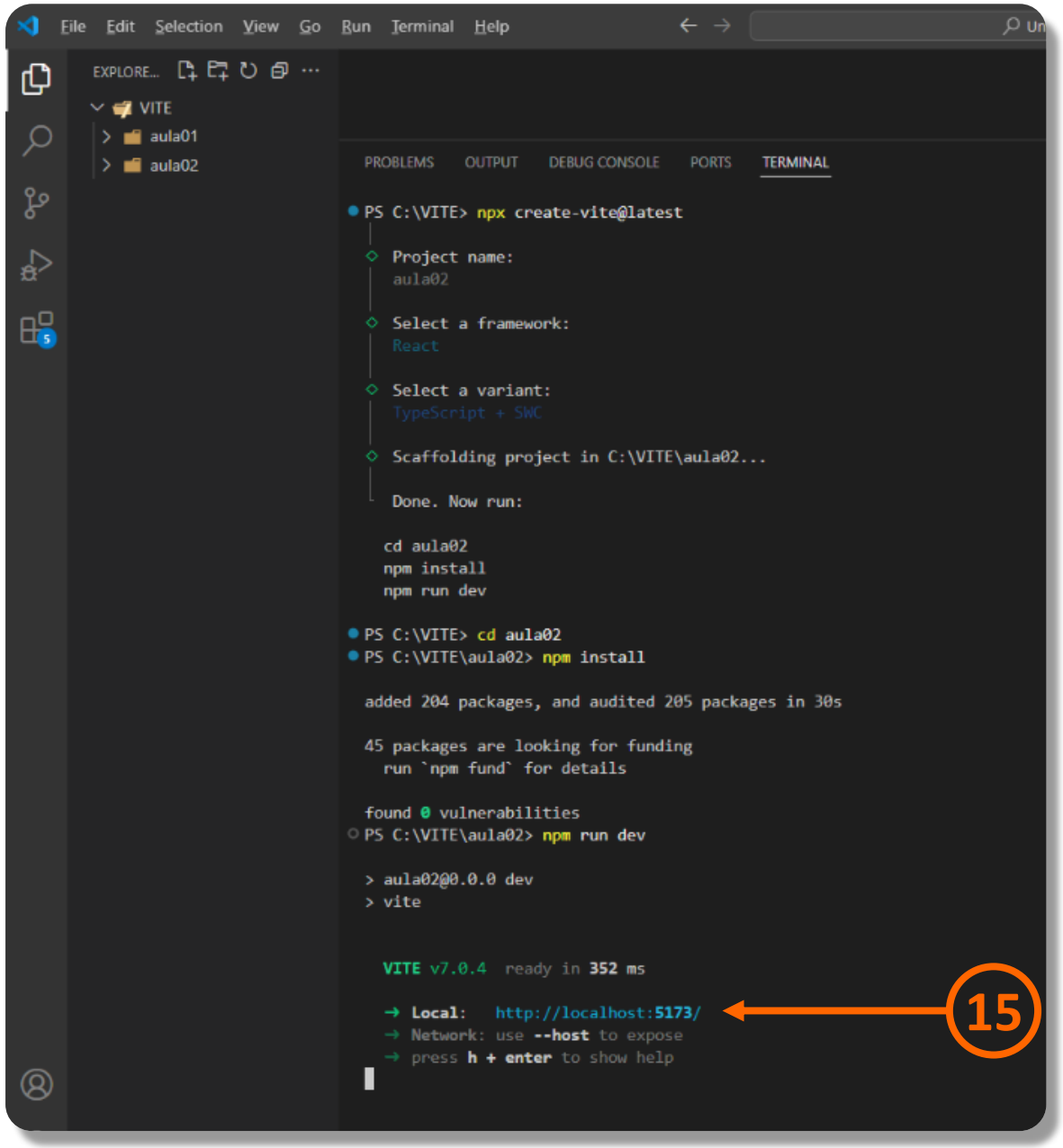


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    15. Pressionamos a tecla CRTL e clicamos na primeira opção.



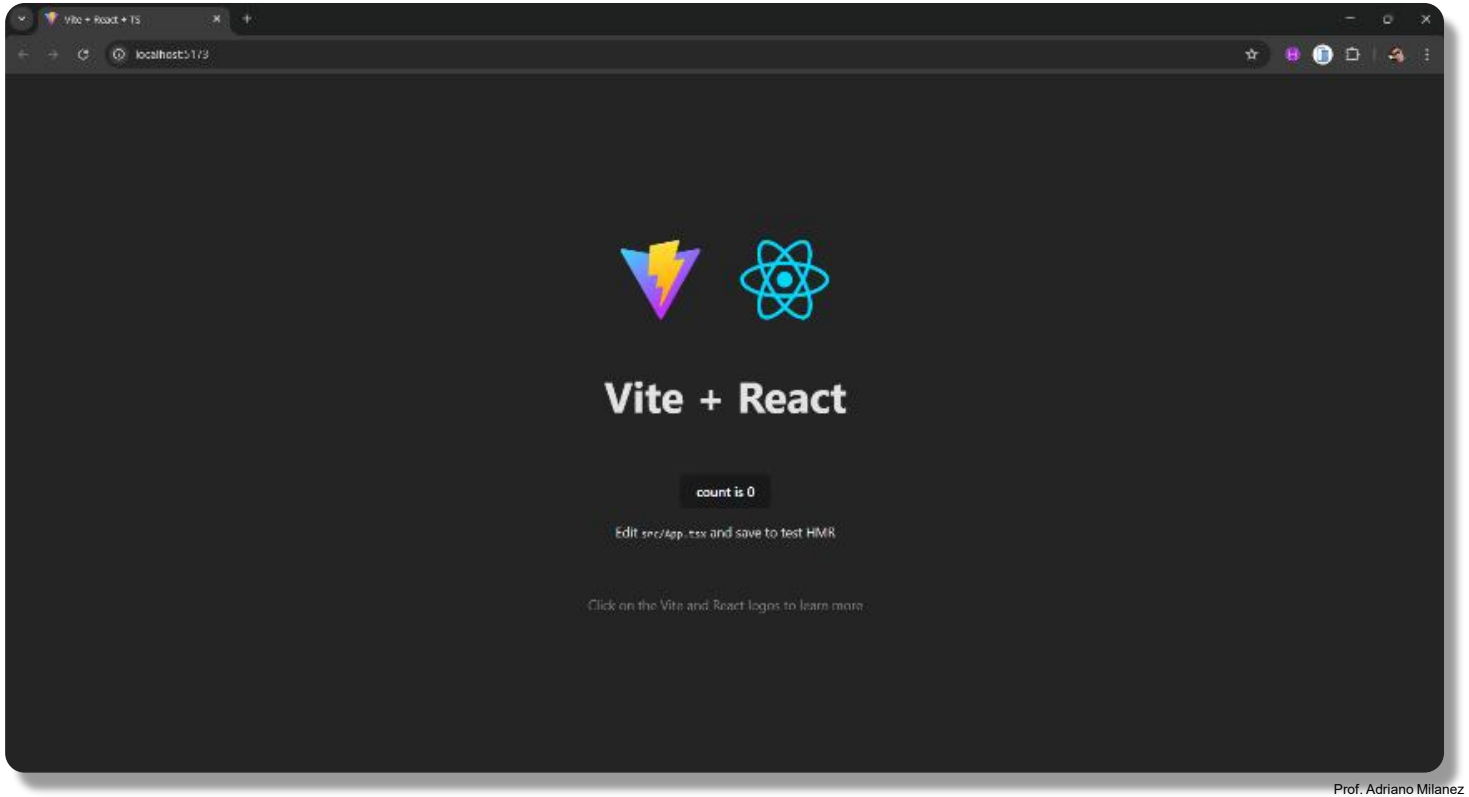
COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos abrir o VS Code.

    16. Como resultado temos.

16

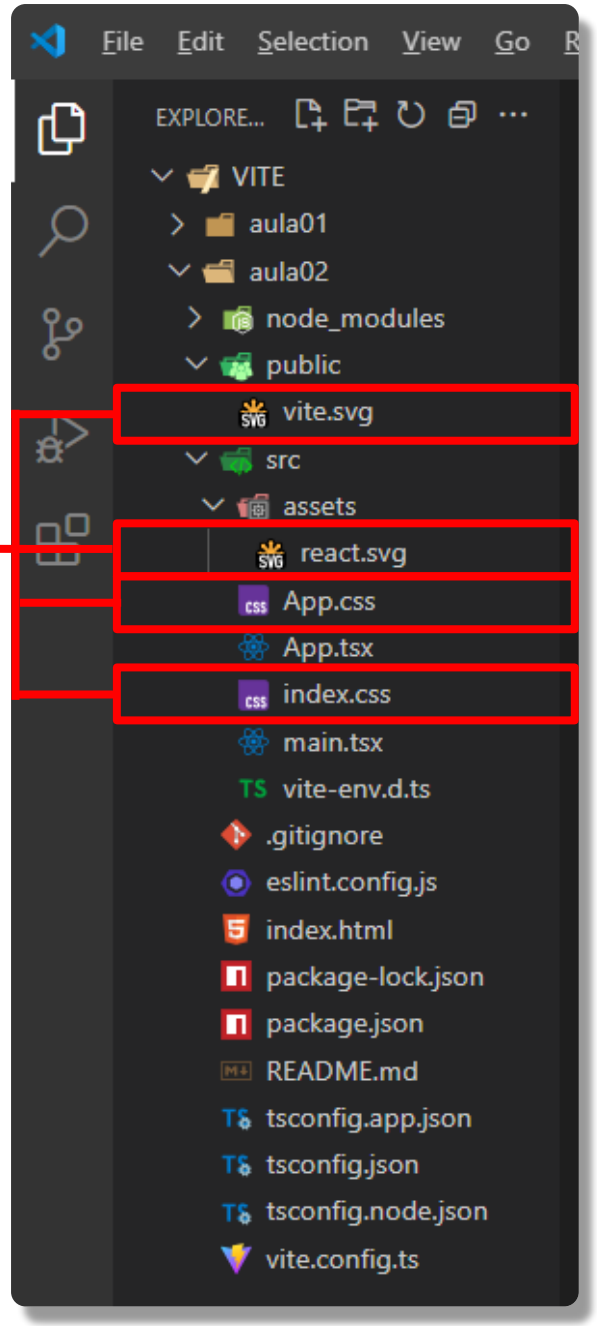


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos apagar arquivos que não usaremos.
    - ✓ Conteúdo da pasta **PUBLIC**
      - ✓ VITE.SVG
    - ✓ Conteúdo da pasta **ASSETS**
      - ✓ REACT.SVG
    - ✓ Arquivos da pasta **SRC**
      - ✓ APP.CSS
      - ✓ INDEX.CSS

Apagamos os arquivos selecionados

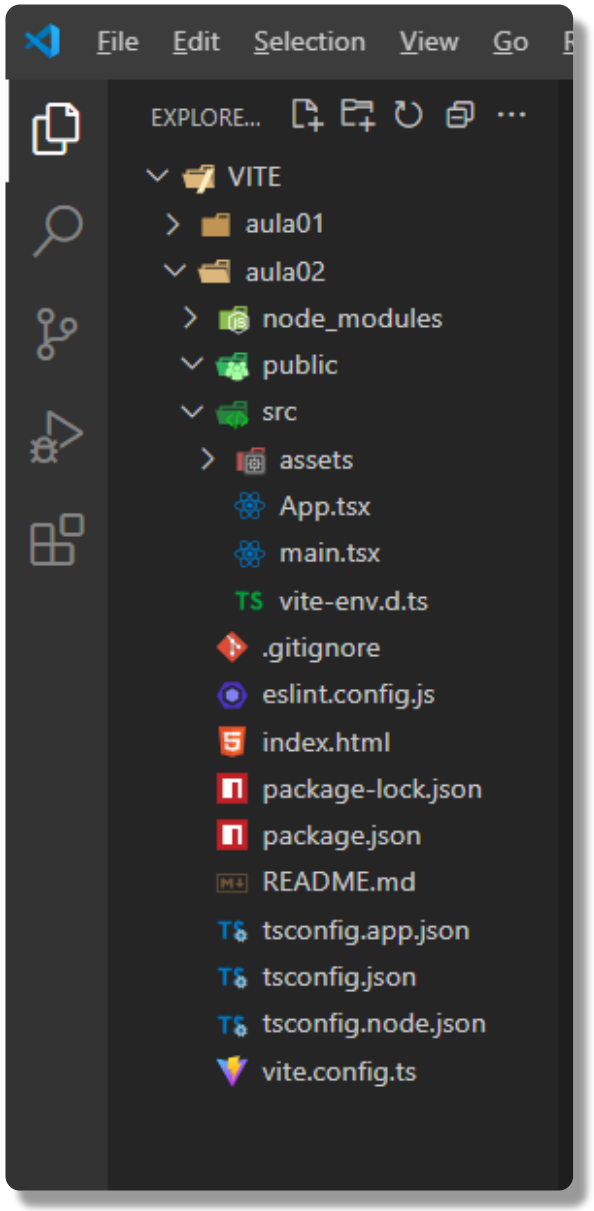


Lembrando que vamos reescrever nossos arquivos MAIN.TSX e modificar o arquivo APP.TSX

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Após excluir os arquivos, teremos em nossa árvores de pastas e arquivos desta forma.



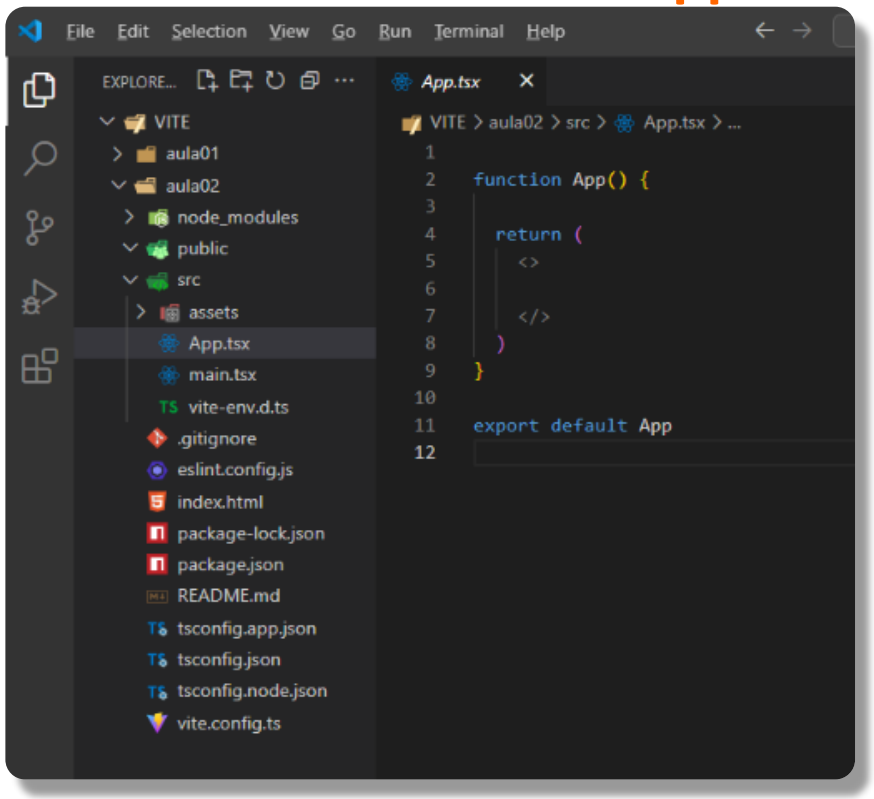
COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos reescrever os arquivos APP.TSX e MAIN.TSX
    - ✓ **APP.TSX**
      - ✓ Apagamos todo o conteúdo e reescrevemos o código.

```
function App() {
    return (
        <>
        </>
    )
}
export default App
```

App.tsx

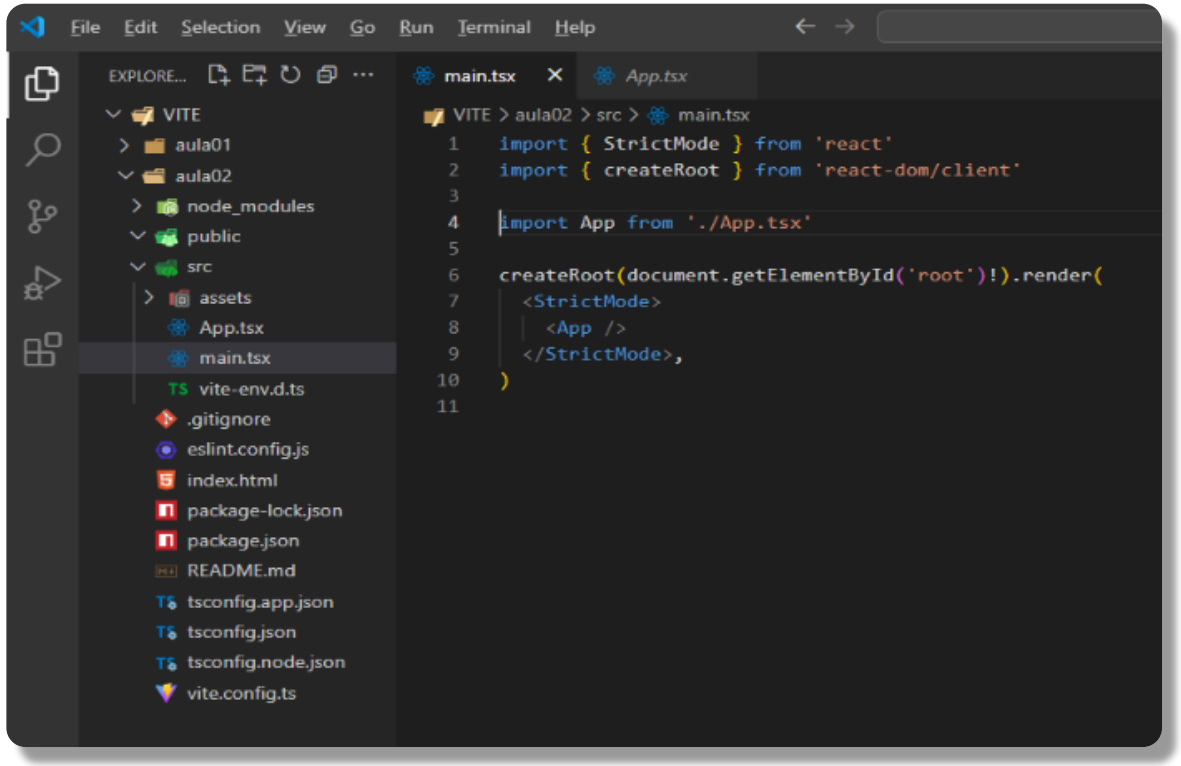


COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos reescrever os arquivos APP.TSX e MAIN.TSX
    - ✓ **MAIN.TSX**
      - ✓ Neste arquivo, apagamos apenas a linha 03, que faz referencia ao arquivo index.css, que apagamos anteriormente.

main.tsx



COMPONENTIZAÇÃO

- **Vamos criar nossa segunda aplicação REACT**

  - Vamos criar nosso primeiro componente reutilizável.
    - Não vamos nos preocupar com coisas que ainda não aprendemos.
    - Onde e como vamos salvar o componente que criaremos?
      - **src/components/Mensagem/Mensagem.tsx**

```
1.  function Mensagem() {
2.      return (
3.          <>
4.              <p>Olá, bem-vindo ao meu site!</p>;
5.          </>
6.      )
7.  }
8.  export default Mensagem;
```

COMPONENTIZAÇÃO

- **Vamos criar nossa segunda aplicação REACT**

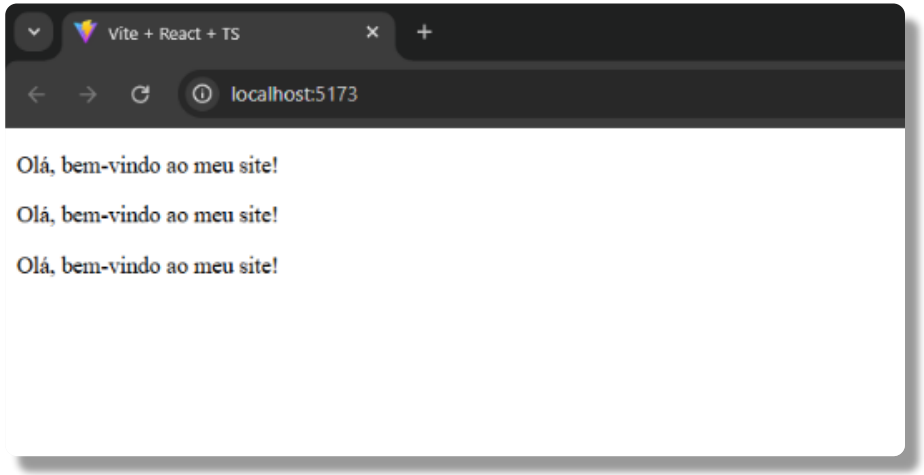
  - Vamos criar nosso primeiro componente reutilizável.
    - No arquivo App.tsx, vamos importar e utilizar o componente Mensagem que acabamos de criar.

```
1.   import Mensagem from "./components/Mensagem/Mensagem"
2.   function App() {
3.       return (
4.           <>
5.               <Mensagem />
6.               <Mensagem />
7.               <Mensagem />
16.          </>
17.      )
18.  }
19.  export default App
```

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos criar nosso primeiro componente reutilizável.
    - ✓ Como resultado de nosso primeiro componente devemos ter a seguinte tela:



COMPONENTIZAÇÃO

- **Vamos criar nossa segunda aplicação REACT**
  - Vamos criar nosso primeiro componente reutilizável.
    - Vamos analisar o código do **Mensagem.tsx** que acabamos de criar.

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

```
function Mensagem() {
/*
    Cria uma função chamada Mensagem.
    Essa função é um componente React, ou seja, uma parte reutilizável da interface.
    O nome começa com letra maiúscula, como recomendado em componentes React.
*/
    return (
    /*
        Tudo o que o componente exibe na tela vai dentro do return.
        O React vai transformar isso em HTML na hora de mostrar no navegador.
    */
        <>
        // é um fragmento React, usado para agrupar elementos sem criar uma nova div.
            <p>Olá, bem-vindo ao meu site!</p>;
            // Aqui, ele retorna um elemento JSX: uma tag <p> com o texto "Olá, bem-vindo ao meu site!".
        </>
    )
}
export default Mensagem;
// Essa linha torna o componente disponível para uso em outros arquivos.
```

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**
  - ✓ Vamos criar nosso primeiro componente reutilizável.
    - ✓ Vamos analisar o código do **App.tsx** que acabamos de modificar.

COMPONENTIZAÇÃO

```
1.  import Mensagem from "./components/Mensagem/Mensagem"
    /*
          Importa o componente Mensagem que está no arquivo Mensagem.tsx, dentro da pasta components/Mensagem.
          Isso permite usar <Mensagem /> dentro do JSX como se fosse uma nova "tag" personalizada.
    */

2.  function App() {
    /*
          Define o componente principal App, que será a base da aplicação React.
          Por padrão, o React inicia renderizando o App.
    */

3.        return (
          /*
               Tudo o que o componente exibe na tela vai dentro do return.
               O React vai transformar isso em HTML na hora de mostrar no navegador.
          */

4.             <>
               // é um fragmento React, usado para agrupar elementos sem criar uma nova div.

5.                  <Mensagem />
                    /*
                         Aqui, estamos chamando o mesmo componente Mensagem três vezes.
                         Cada vez que <Mensagem /> aparece, ele renderiza o conteúdo definido em Mensagem.tsx,
                    */

…                   …

7.                  <Mensagem />

16.            </>

17.       )

18. }

19. export default App
    // Exporta o componente App para que ele possa ser usado por outros arquivos — geralmente por main.tsx, que o renderiza na tela.
```

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos criar nosso segundo componente reutilizável.
    - ✓ Não vamos nos preocupar com coisas que ainda não aprendemos.
    - ✓ Onde e como vamos salvar o componente que criaremos?
      - ✓ **src/components/Botao/Botao.tsx**

COMPONENTIZAÇÃO

```
1.  function Botao({ texto }: { texto: string }) {
2.      const estilo = {
3.          backgroundColor: "#4f46e5",
4.          color: "white",
5.          padding: "0.5rem 1rem",
6.          border: "none",
7.          borderRadius: "6px",
8.          fontSize: "1rem",
9.          cursor: "pointer",
10.     };
11.     return(
12.         <>
13.             <button style={estilo}>{texto}</button>
14.         </>
15.     )
16. }
17. export default Botao;
```

COMPONENTIZAÇÃO

- **Vamos criar nossa segunda aplicação REACT**

  - Vamos criar nosso segundo componente reutilizável.

    - No arquivo **App.tsx**, vamos importar o componente botão que acabamos de criar e reutilizá-lo.

COMPONENTIZAÇÃO

```
1.  import Botao from "./components/Mensagem/Mensagem"
2.  import Botao from "./components/Botao/Botao"
3.  function App() {
4.      return (
5.          <>
6.              <Mensagem />
7.              <Mensagem />
8.              <Mensagem />
9.              <Botao texto="Clique aqui" />
10.             <Botao texto="Enviar" />
11.             <Botao texto="Cancelar" />
12.             <Botao texto="Salvar" />
13.             <Botao texto="Excluir" />
14.             <Botao texto="Atualizar" />
15.             <Botao texto="Carregar" />
16.             <Botao texto="Pesquisar" />
17.         </>
18.     )
19. }
20. export default App
```

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ Vamos criar nosso segundo componente reutilizável.
    - ✓ Como resultado de nosso primeiro componente devemos ter a seguinte tela:

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**
  - ✓ Vamos analisar o código do **Botao.tsx** que acabamos de criar.

COMPONENTIZAÇÃO

```
1.  function Botao({ texto }: { texto: string }) {
    // Declara uma função chamada Botao, que é um componente React.
    // Ele recebe uma props chamada texto, e garante que texto é uma string (graças ao TypeScript).
    // Não vamos nos preocupar em entender o que é um props, pois veremos isto mais tarde.
2.      const estilo = {
…           …
10.     };
        // Cria um objeto chamado estilo com as propriedades CSS para o botão.
        // Esse estilo será aplicado diretamente no botão como CSS inline.
        // As propriedades controlam cor de fundo, texto, espaçamento, bordas, etc.
11.     return(
        // Define o que o componente vai exibir na tela.
12.         <>
            // é um fragmento React, usado para agrupar elementos sem criar uma nova div.
13.             <button style={estilo}>{texto}</button>
                // Cria um botão e aplica os estilos inline.
                // Insere o conteúdo vindo da props dentro do botão.
14.         </>
15.     )
16. }
17. export default Botao;
    // Torna o componente Botao disponível para ser importado e usado em outros arquivos do projeto.
```

COMPONENTIZAÇÃO

- ✓ **Vamos criar nossa segunda aplicação REACT**
  - ✓ Vamos analisar o código do **App.tsx** que acabamos de criar.

COMPONENTIZAÇÃO

```
1.  import Botao from "./components/Mensagem/Mensagem"  // Importa o componente Mensagem localizado na pasta components/Mensagem/Mensagem.tsx.
2.  import Botao from "./components/Botao/Botao"  // Importa o componente Botao localizado na pasta components/Botao/Botao.tsx.
3.  function App() {  // Declara o componente principal da aplicação, chamado App.
4.      return (  //  O componente App retorna um JSX (código que parece HTML).
5.          <>  //  É um fragmento React, que permite agrupar múltiplos elementos sem adicionar uma div desnecessária ao DOM.
6.              <Mensagem />
                /*
                    Aqui, estamos chamando o mesmo componente Mensagem três vezes.
                    Cada vez que <Mensagem /> aparece, ele renderiza o conteúdo definido em Mensagem.tsx,
                */
…               …
9.              <Botao texto="Clique aqui" />
                /*
                  São 08 chamadas ao componente <Botao />, cada uma com uma prop texto diferente.
                  Isso demonstra reutilização de componente: o mesmo botão é exibido com textos diferentes.
                  O valor passado em texto="..." será exibido dentro do botão, de acordo com a lógica definida em Botao.tsx.
                */
…               …
16.             <Botao texto="Pesquisar" />
17.         </>
18.     )
19. }
20. export default App //  Exporta o componente App, tornando-o visível para o main.tsx, onde geralmente ele é usado como o componente raiz da aplicação React.
```

COMPONENTIZAÇÃO – INSERINDO IMAGENS

- ✓ **Vamos criar nossa segunda aplicação REACT**

  - ✓ No mesmo componente APP.TSX, vamos inserir imagens.
  - ✓ Existe **duas** formas para a inserção de imagens:
    1. **Imagens estáticas**, que não mudam
       - ✓ **Onde ficam**
         - ✓ As imagens ficam armazenadas na pasta PUBLIC/IMAGENS
       - ✓ **Como usar**
         - ✓ `<img src="/images/logo.png" alt="Logo" />`
    2. **Imagens dinâmicas**, utilizadas para trocas dinâmicas
       - ✓ **Onde ficam**
         - ✓ As imagens ficam armazenadas na pasta SRC/ASSETS/IMAGENS
       - ✓ **Como usar**
         - ✓ `import logo from './assets/logo.png';`
         - ✓ `<img src={logo} alt="Logo" />`

COMPONENTIZAÇÃO – INSERINDO IMAGENS

- ✓ **Vamos criar nossa segunda aplicação REACT**

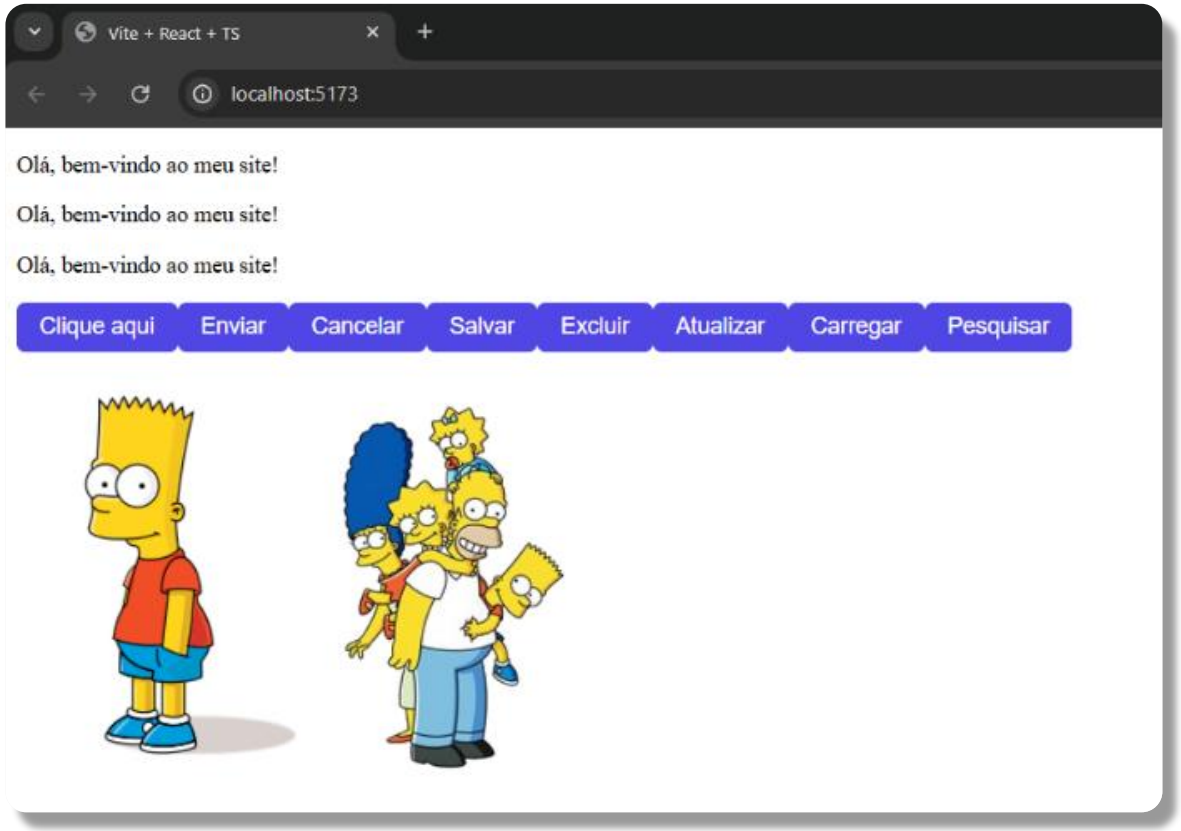
  - ✓ No mesmo componente APP.TSX, vamos inserir imagens.
  - ✓ Existe duas formas para a inserção de imagens:
    1. **Imagens estáticas**, que não mudam
    2. **Imagens dinâmicas**, utilizadas para trocas dinâmicas
  - ✓ Vamos inserir no componente **APP.TSX** as duas formas

COMPONENTIZAÇÃO

```
1.  import Botao from "./components/Mensagem/Mensagem"
2.  import Botao from "./components/Botao/Botao"
3.
4.  import simpsons from "./assets/imagens/simpsons.png"
5.
6.  function App() {
7.      return (
8.          <>
9.              <Mensagem />
…               …
12.             <Botao texto="Clique aqui" />
…               …
19.             <Botao texto="Pesquisar" />
20.             <br />
21.             <img src="/imagens/bart.png" alt="Bart" />
22.             <img src={ simpsons } alt="Simpsons" />
23.         </>
24.     )
25. }
26. export default App
```

COMPONENTIZAÇÃO

- ✓ Vamos criar nosso segundo componente reutilizável.
  - ✓ Como resultado da inserção das imagens devemos ter a seguinte tela:



2

VITE – COMPONENTIZAÇÃO – EXERCÍCIOS

VITE – EXERCÍCIOS

- **Exercício 01 - Criando o componente Alerta**
  - **Objetivo:**
    - Praticar a criação de um componente React simples, sem o uso de props, com uma mensagem de alerta visual.
  - **Instruções**:
    1. Crie uma pasta chamada Alerta dentro da pasta src/components.
    2. Dentro da pasta Alerta, crie o arquivo Alerta.tsx.
    3. O componente deve exibir uma mensagem de aviso estilizada com texto em vermelho e em negrito.
    4. No arquivo App.tsx, importe e utilize o componente Alerta pelo menos duas vezes.

VITE – EXERCÍCIOS

- **Exercício 02 - Criando o componente TituloSecundario**
  - **Objetivo:**
    - Praticar a criação de um componente básico em React, focando na reutilização de estrutura fixa.
  - **Instruções:**
    1. Na pasta src/components, crie uma nova subpasta chamada TituloSecundario.
    2. Dentro dessa subpasta, crie o arquivo TituloSecundario.tsx.
    3. O componente deve exibir um subtítulo (<h2>) com o texto: “Esta é uma seção complementar do site”
    4. No arquivo App.tsx, importe e utilize o componente TituloSecundario duas vezes, entre outros componentes que já tenha criado.

VITE – EXERCÍCIOS

- **Exercício 02 - Criando Componentes Simples com React**
  - Praticar a criação de **componentes reutilizáveis** em React, **sem o uso de props**, organizando o projeto de forma modular.
    - **Instruções**:
      - Crie uma aplicação React com Vite e TypeScript.
      - Dentro da pasta src, crie uma nova pasta chamada components.
        - Dentro da pasta components, crie duas subpastas:
          - Cabecalho
          - Mensagem
        - Em cada subpasta, crie um arquivo .tsx com o nome do componente:
          - Cabecalho.tsx: deve retornar um título (<h1>) com o texto “Meu Primeiro Site em React”.
          - Mensagem.tsx: deve retornar um parágrafo (<p>) com o texto “Este é um exemplo simples de componente.”
        - No arquivo App.tsx, importe os dois componentes e utilize-os da seguinte forma:
          - O Cabecalho deve aparecer no topo da página.
          - O Mensagem deve ser usado três vezes, logo abaixo do cabeçalho.

VITE – EXERCÍCIOS

- **Exercício 02 - Componente de Rodapé**
  - Criar um componente chamado Rodape que exibe uma mensagem fixa no final da página. O foco é reforçar a estrutura e o uso de componentes.
    - **Instruções**:
      - Na mesma aplicação React que você já criou, vá até a pasta src/components.
        - Crie uma nova subpasta chamada Rodape.
          - Dentro dela, crie o arquivo Rodape.tsx.
          - O componente Rodape deve retornar um parágrafo com o texto:
            - © 2025 - Meu Site React. Todos os direitos reservados.
  - **Desafio**
    - Insira imagens.
    - Crie o CSS para formatar, use como base nos exemplos criado em aula.

REFERÊNCIAS BIBLIOGRÁFICAS

COMPONENTIZAÇÃO

ANTONIO, C. Pro React: Build Complex Front-End Applications in a Composable Way With React. Apress, 2015.

BOSWELL, D; FOUCHER, T. The Art of Readable Code: Simple and Practical Techniques for Writing Better Code. Estados Unidos: O'Reilly Media, 2012.

BRITO, Robin Cris. Android Com Android Studio - Passo A Passo. Editora Ciência Moderna.

BUNA, S. React Succinctly. Estados Unidos: [s.n], 2016. Disponível em: <www.syncfusion.com/ebooks/reactjs_succinctly>. Acesso em: 10 de julho de 2025.

FACEBOOK (2019a). React: Getting Started. React Docs, 2019. Disponível em: <reactjs.org/docs/react-api.html>. Acesso em: 10 de julho de 2025.

FACEBOOK (2019b). React Without ES6. React Docs, 2019. Disponível em: <reactjs.org/docs/react-without-es6.html>. Acesso em: 10 de julho de 2025.

FACEBOOK (2019c). React Without JSX. React Docs, 2019. Disponível em: <reactjs.org/docs/react-without-jsx.html>. Acesso em: 10 de julho de 2025.

FREEMAN, Eric ROBSON, Elisabeth. Use a Cabeça! Programação em HTML5. Rio de Janeiro: Editora Alta Books, 2014

GACKENHEIMER, C. Introduction to React: Using React to Build scalable and efficient user interfaces.[s.i.]: Apress, 2015.

HUDSON, P. Hacking with React. 2016. Disponível em: <www.hackingwithreact.com/read/1/3/introduction-to-jsx>. Acesso em: 10 de julho de 2025.

KOSTRZEWA, D. Is React.js the Best JavaScript Framework in 2018? 2018. Disponível em: <hackernoon.com/is-react-js-the-best-JavaScript-framework-in-2018-264a0eb373c8>. Acesso em: 10 de julho de 2025.

MARTIN, R. Clean Code: A Handbook of Agile Software Craftsmanship. Estados Unidos: Prentice Hall, 2009.

MDN WEB DOCS. Guia JavaScript. Disponível em <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide>. Acessado em 10 de julho de 2025.

Referências Bibliográficas

Componentização

NELSON, J. Learn React’s Fundamentals Without the Buzzwords? 2018. Disponível em: <jamesknelson.com/learn-react-fundamentals-sans-buzzwords>. Acesso em: 10 de julho de 2025.

NIELSEN, J. Response Times: The 3 Important Limits. 1993. Disponível em: <www.nngroup.com/articles/response-times-3-important-limits>. Acesso em: 10 de julho de 2025.

O'REILLY, T. What is Web 2.0: Design Patterns and Business Models for the Next Generation of Software. 2005. Disponível em: <www.oreilly.com/pub/a/web2/archive/what-is-web-20.html#mememap>. Acesso em: 10 de julho de 2025.

PANDIT, N. What Is ReactJS and Why Should We Use It? 2018. Disponível em: <www.c-sharpcorner.com/article/what-and-why-reactjs>. Acesso em: 10 de julho de 2025.

RAUSCHMAYER, A. Speaking JavaScript: An In-Depth Guide for Programmers. Estados Unidos: O’Reilly Media, 2014.

REACTIVA. O arquivo package-lock.json. Disponível em: <https://nodejs.reativa.dev/0020-package-lock-json/index>. Acessado em 10 de julho de 2025.

_____. O guia do package.json. Disponível em: <https://nodejs.reativa.dev/0019-package-json/index>. Acessado em 10 de julho de 2025.

RICOY, L. Desmitificando React: Uma Reflexão para Iniciantes. 2018. Disponível em: <medium.com/trainingcenter/desmitificando-react-uma-reflex%C3%A3o-para-iniciantes-a57af90b6114>. Acesso em: 10 de julho de 2025.

SILVA, Maurício Samy. Ajax com jQuery: requisições Ajax com a simplicidade de jQuery. São Paulo: Novatec Editora, 2009.

_____. Construindo Sites com CSS e XHTML. Sites Controlados por Folhas de Estilo em Cascata. São Paulo: Novatec, 2010.

_____. CSS3 - Desenvolva aplicações web profissionais com o uso dos poderosos recursos de estilização das CSS. São Paulo: Novatec Editora, 2010.

STACKOVERFLOW. Most Popular Technologies: Web Frameworks. Developer Survey Results, StackOverflow, 2019. Disponível em: <insights.stackoverflow.com/survey/2019#technology>. Acesso em: 10 de julho de 2025.

Componentização

Referências Bibliográficas

W3C. HTML5 - A linguagem de marcação que revolucionou a web. São Paulo: Novatec Editora, 2010.

_____. A vocabulary and associated APIs for HTML and XHTML. Disponível em <https://www.w3.org/TR/2018/SPSD-html5-20180327/>. Acessado em 10 de julho de 2025.

_____. Cascading Style Sheets, level 1. Disponível em <https://www.w3.org/TR/2018/SPSD-CSS1-20180913/>. Acessado em 10 de julho de 2025.

_____. Cascading Style Sheets, level 2 Revision 2. Disponível em <https://www.w3.org/TR/2016/WD-CSS22-20160412/>. Acessado em 10 de julho de 2025.

_____. Cascading Style Sheets, level 2. Disponível em <https://www.w3.org/TR/2008/REC-CSS2-20080411/>. Acessado em 10 de julho de 2025.

_____. Cascading Style Sheets, level 3. Disponível em <https://www.w3.org/TR/css-syntax-3/>. Acessado em 10 de julho de 2025.

_____. HTML 3.2 Reference Specification. Disponível em <https://www.w3.org/TR/2018/SPSD-html32-20180315/>. Acessado em 10 de julho de 2025.

_____. HTML 4.0 Specification. Disponível em <https://www.w3.org/TR/1998/REC-html40-19980424/>. Acessado em 10 de julho de 2025.

_____. HTML 4.01 Specification. Disponível em <https://www.w3.org/TR/2018/SPSD-html401-20180327/>. Acessado em 10 de julho de 2025.

_____.Cascading Style Sheets, level 2 Revision 1. Disponível em <https://www.w3.org/TR/CSS2/>. Acessado em 10 de julho de 2025.

WIKIPEDIA. JavaScript. Disponível em <https://pt.wikipedia.org/wiki/JavaScript>. Acessado em 10 de julho de 2025.